Catalan's Numbers

A binary bracketing is a bracketing built up entirely of binary operations. The number of binary bracketings of n letters (Catalan's problem) are given by the Catalan numbers C_{n-1}, where

$$C_n \equiv \frac{1}{n+1}\binom{2n}{n} \tag{1}$$

$$= \frac{1}{n+1}\frac{(2n)!}{n!^2} = \frac{(2n)!}{(n+1)!\,n!}, \tag{2}$$

where $\binom{2n}{n}$ denotes a binomial coefficient and $n!$ is the usual factorial, as first shown by Catalan in 1838. For example, for the four letters a, b, c, and d there are five possibilities: $((a\,b)\,c)\,d$, $(a\,(b\,c))\,d$, $(a\,b)\,(c\,d)$, $a\,((b\,c)\,d)$, and $a\,(b\,(c\,d))$, written in shorthand as $((x\,x)\,x)\,x$, $(x\,(x\,x))\,x$, $(x\,x)\,(x\,x)$, $x\,((x\,x)\,x)$, and $x\,(x\,(x\,x))$.

The Catalan numbers satisfy the recurrence relations[1]

```
const int MOD = ....
const int MAX = ....
int catalan[MAX];
void init() {
   catalan[0] = catalan[1] = 1;
   for (int i=2; i<=n; i++) {
      catalan[i] = 0;
      for (int j=0; j < i; j++) {
         catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
         if (catalan[i] >= MOD) {
            catalan[i] -= MOD;
         }
      }
   }
}
```

c0=1, c(n+1)=for i=0 to n, summation of all c(i)*c(n-i)

and also c(n+1) = (2(2*n+1)) / (n+2) *c(n)

Super Catalan's Numbers

While the Catalan numbers are the number of *p*-good paths from (n, n) to (0,0) which do not cross the diagonal line, the super Catalan numbers count the number of lattice paths with diagonal steps from (n, n) to (0,0) which do not touch the diagonal line $x = y$.

The super Catalan numbers are given by the recurrence relation

$$S(n) = \frac{3(2n-3)\,S(n-1) - (n-3)\,S(n-2)}{n}$$

Anagrammatic Primes

Only this numbers are anagrammatic primes

2, 3, 5, 7, 11, 13, 17, 31, 37, 71, 73, 79, 97, 113, 131, 199, 311, 337, 373, 733, 919, 991

Auler path

অয়লার পাথ হলো গ্রাফের এমন একটি পাথ যেটা যেকোনো একটি নোডে শুরু হয়ে অন্য একটি নোডে শেষ হয়েছে কিন্তু প্রতিটি এজ ঠিক একবার ব্যবহার করেছে।

Directed Graph: যদি গ্রাফটি কানেক্টেড হয় এবং ২টি মাত্র নোডের আউটডিগ্রি এবং ইনডিগ্রির পার্থক্য 1 হয় এবং বাকি সব নোডের আউটডিগ্রি এবং ইনডিগ্রি সমান হয়। শুরুর নোডের ক্ষেত্রে indegree=outdegree–1 আর শেষের নোডের ক্ষেত্রে outdegree=indegree–1 হবে।

Undirected Graph: যদি গ্রাফটি কানেক্টেড হয় এবং ২টি মাত্র নোডের ডিগ্রি বিজোড় সংখ্যা হয় শুরু) হয়। সংখ্যা জোড় ডিগ্রি নোডের সব বাকি এবং (নোড শেষের এবং

Auler Circuit

অয়লার সার্কিট হলো গ্রাফের এমন একটা পাথ যেটা যে নোডে শুরু হয়েছে সে নোডেই শেষ হয়েছে কিন্তু প্রতিটি এজ ঠিক একবার ব্যবহার করেছে।

Directed Graph: অয়লার সার্কিট থাকবে যদি গ্রাফটি কানেক্টেড হয় এবং প্রতিটি নোডের আউটডিগ্রি এবং ইনডিগ্রি সমান হয়।

Undirected Graph: যদি গ্রাফটি কানেক্টেড হয় এবং প্রতিটি নোডের ডিগ্রী জোড় সংখ্যা হয়।

```
#include<iostream>
#include<vector>
#define NODE 5
using namespace std;

int graph[NODE][NODE] = {
  {0, 1, 1, 1, 1},
  {1, 0, 1, 1, 0},
  {1, 1, 0, 1, 0},
  {1, 1, 1, 0, 1},
  {1, 0, 0, 1, 0}
};

int tempGraph[NODE][NODE];

int findStartVert() {
  for(int i = 0; i<NODE; i++) {
    int deg = 0;
    for(int j = 0; j<NODE; j++) {
      if(tempGraph[i][j])
        deg++;    //increase degree, when connected edge found
    }
```

```
    if(deg % 2 != 0)     //when degree of vertices are odd
      return i;        //i is node with odd degree
  }
  return 0;     //when all vertices have even degree, start from 0
}

bool isBridge(int u, int v) {
  int deg = 0;
  for(int i = 0; i<NODE; i++)
    if(tempGraph[v][i])
      deg++;

  if(deg>1) {
    return false;    //the edge is not forming bridge
  }

  return true;   //edge forming a bridge
}

int edgeCount() {
  int count = 0;
  for(int i = 0; i<NODE; i++)
    for(int j = i; j<NODE; j++)
      if(tempGraph[i][j])
        count++;
  return count;     //count nunber of edges in the graph
}

void fleuryAlgorithm(int start) {
  static int edge = edgeCount();
  for(int v = 0; v<NODE; v++) {
    if(tempGraph[start][v]) {       //when (u,v) edge is presnt and not forming bridge
      if(edge <= 1 || !isBridge(start, v)) {
        cout << start << "--" << v << " ";
        tempGraph[start][v] = tempGraph[v][start] = 0;     //remove edge from graph
        edge--;    //reduce edge
        fleuryAlgorithm(v);
      }
    }
  }
}

int main() {
```

```
  for(int i = 0; i<NODE; i++)     //copy main graph to tempGraph
    for(int j = 0; j<NODE; j++)
      tempGraph[i][j] = graph[i][j];
  cout << "Euler Path Or Circuit: ";
  fleuryAlgorithm(findStartVert());
}
```

SPARSE Table

```
#include<bits/stdc++.h>
using namespace std;
#define mx -1e9
const int k = 16;
const int N = 1e5;

int n,q,l,r;
int table[N][k+1],arr[N];

int main()
{
   //freopen("f.txt","r",stdin);
   cin>>n>>q;
   for(int i=0; i<n; i++)
   {
      cin>>arr[i];
      table[i][0]=arr[i];
   }
   /// Build Sparse Table
   for(int j=1; j<=k; j++)
   {
      for(int i=0; i<=n-(1<<j); i++)
      {
         table[i][j]=max(table[i][j-1],table[i+(1<<(j-1))][j-1]);
      }
   }

   ///query

   while(q--)
   {
      cin>>l>>r;
      l--;
```

```
        r--;
        int ans=mx;
        for(int j=k;j>=0;j--)
        {
            if(l+(1<<j)-1<=r)
            {
                ans=max(ans,table[l][j]);
                l+=(1<<j);
            }
        }
        cout<<ans<<endl;
    }
    return 0;
}
```

Maximal Bipartite Matching

```
#include <iostream>
#include <string.h>
using namespace std;

// M is number of applicants
// and N is number of jobs
int N,M;
int bpGraph[501][501];
/// An array to keep track of the applicants assigned to jobs.
/// The value of matchR[i] is the applicant number assigned to job i,
/// the value -1 indicates nobody is assigned.
int matchR[501];
bool seen[501];

/// A DFS based recursive function that returns true if a matching for vertex u is possible

bool bpm(int u)
{
    // Try every job one by one
    for (int v = 0; v < N; v++)
    {
        /// If applicant u is interested in job v and v is not visited
        if (bpGraph[u][v] && !seen[v])
        {
            // Mark v as visited
            seen[v] = true;
            ///If job 'v' is not assigned to an applicant OR previously assigned applicant for job v
            ///(which is matchR[v]) has an alternate job available.Since v is marked as visited in the
```

```
        ///above line, matchR[v] in the following recursive call will not get job 'v' again
        if (matchR[v] < 0 || bpm(matchR[v]))
        {
           matchR[v] = u;
           return true;
        }
     }
  }
  return false;
}
// Returns maximum number of matching from M to N
int maxBPM()
{
  // Initially all jobs are available
  memset(matchR, -1, sizeof(matchR));
  // Count of jobs assigned to applicants
  int result = 0;
  for (int u = 0; u < M; u++)
  {
     /// Mark all jobs as not seen for next applicant.
     memset(seen, 0, sizeof(seen));
     // Find if the applicant 'u' can get a job
     if (bpm(u))
        result++;
  }
  return result;
}

int main()
{
  scanf("%d%d",&N,&M);
  for(int i=0; i<N; i++)
     for(int j=0; j<M; j++)
        scanf("%d",&bpGraph[i][j]);
  cout << "Maximum number of applicants that can get job is "<< maxBPM();
  return 0;
}
```

Maximum Flow (Ford Fulkerson algorithm)

```
#define max_node 101

int n,m,u,v,w,s,t; //s means source and t means sink
vector<int>adj[max_node];
```

```
int risidual_graph[max_node][max_node],parent[max_node];

bool bfs()
{
   bool visited[n+1];
   memset(visited, 0, sizeof(visited));
   queue<int>q;
   q.push(s);
   visited[s]=1;
   parent[s]=-1;
   while(!q.empty())
   {
      u=q.front();
      q.pop();
      for(int i=0;i<adj[u].size();i++)
      {
         v=adj[u][i];
         if(visited[v]==false&&risidual_graph[u][v]>0)
         {
            q.push(v);
            parent[v]=u;
            visited[v]=true;
         }
      }
   }
   return visited[t];
}

int fordFulkerson()
{
   int max_flow=0;
   while(bfs())
   {
      int path_flow=INT_MAX;
      v=t;
      while(v!=s)
      {
         u=parent[v];
         path_flow=min(path_flow,risidual_graph[u][v]);
         v=parent[v];
      }
      v=t;
      while(v!=s)
      {
```

```
        u=parent[v];
        risidual_graph[u][v]-=path_flow;
        risidual_graph[v][u]+=path_flow;
        v=parent[v];
      }
      max_flow+=path_flow;
   }
   return max_flow;
}

int main()
{
   freopen("f.txt","r",stdin);
   //cout<<"Enter node number and edge number: ";
   cin>>n>>m;
   //cout<<"Enter edges:\n";
   for(int i=0;i<m;i++)
   {
      cin>>u>>v>>w;
      adj[u].push_back(v);
      risidual_graph[u][v]=w;
   }
   cin>>s>>t;
   cout<<"The maximum possible flow is "<<fordFulkerson()<<endl;
   return 0;
}
```

Digit DP

```
/// How many numbers x are there in the range a to b where the
/// digit d occurs exactly k times in x?
#include<bits/stdc++.h>
using namespace std;

int a,b,d,k,num_size;
vector<int>num;
int dp[15][15][15];

/// DP[p][c][f] = Number of valid numbers <= b from this state
/// p = current position from left side (zero based)
/// c = number of times we have placed the digit d so far
```

```
/// f = the number we are building has already become smaller than b? [0 = no, 1 = yes]

int call(int pos,int cnt,int f)
{
   if(cnt>k)
      return 0;
   if(pos==num_size)
   {
      if(cnt==k)
         return 1;
      return 0;
   }
   if(dp[pos][cnt][f]!=-1)
      return dp[pos][cnt][f];
   int lmt;
   if(f==0)
      lmt=num[pos];
   else
      lmt=9;
   int tf,tcnt,res=0;
   /// Try to place all the valid digits such that the number doesn't exceed b
   for(int i=0;i<=lmt;i++)
   {
      tf=f;
      tcnt=cnt;
      if(f==0&&i<lmt)
         tf=1;   /// The number is getting smaller at this position
      if(i==d)
         tcnt++;
      if(tcnt<=k)
         res+=call(pos+1,tcnt,tf);
   }
   return dp[pos][cnt][f]=res;
}

int solve(int x)
{
   num.clear();
   while(x>0)
   {
      num.push_back(x%10);
      x/=10;
   }
   num_size=num.size();
```

```
   /// Stored all the digits of b in num for simplicity
   reverse(num.begin(),num.end());
   memset(dp,-1,sizeof dp);
   return call(0,0,0);
}

int main()
{
   cin>>a>>b>>d>>k;
   int ans=solve(b)-solve(a-1);
   cout<<ans<<endl;
   return 0;
}
```

Turán's Theorem: Let *G* be any graph with *n* vertices, such that *G* is K_{r+1} -free. Then the number of edges in *G* is at most
((r-1)/r) *((n*n)/2) = ((r-1) * n^2) / (r*2)

Turán's Theorem (Second Formulation). Among the *n*-vertex simple graphs with no (*r* + 1)-cliques, *T*(*n*, *r*) has the maximum number of edges.
In the mathematical area of graph theory, a clique is a subset of vertices of an undirected graph such that every two distinct vertices in the clique are adjacent; that is, its induced subgraph is complete.
uran's theorem states the following:
If a graph with nn vertices does not contain a clique of size r+1r+1, then the number of edges in that graph is at
most 12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2)12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2).
The former statement is equivalent to the following statement:
If a graph with nn vertices has more
than 12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2)12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2)edges, then it does contain a clique of size r+1r+1.
The minimum size of the largest clique in any graph with n≥2n≥2 vertices and m≥1m≥1 edges can then be computed in a naive way as follows:
(1.) set r=n−1r=n−1
(2.)
compute m∗(r)=12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2)m∗(r)=12(n2−(nmodr)⌈n/r⌉2−(r−(nmodr))⌊n/r⌋2)
(3.) if m>m∗(r)m>m∗(r) return r+1r+1 else decrement rr by 1 and go to (2.)
The algorithm above runs in linear time. A more efficient solution runs in logarithmic time. The idea is to utilize binary search to find the value r+1r+1 such
that m>m∗(r)m>m∗(r) and m≤m∗(r+1)m≤m∗(r+1).

Matrix Exponentiation
///Solve F(n)=a*F(n-1)+b*F(n-2)+c*F(n-3) for n>=3
Putting a = 1, b = 1 and c = 1 in above formula

```
| F(n)   |  =  [ | 1 1 1 | ] ^ (n-2)   *   | F(2) |
| F(n-1) |     [ | 1 0 0 | ]             | F(1) |
| F(n-2) |     [ | 0 1 0 | ]             | F(0) |
```

```
#include<bits/stdc++.h>
using namespace std;

void multiply(int a[3][3],int b[3][3])
{
   int mul[3][3];
   for(int i=0; i<3; i++)
   {
      for(int j=0; j<3; j++)
      {
         mul[i][j]=0;
         for(int k=0; k<3; k++)
            mul[i][j]+=a[i][k]*b[k][j];
      }
   }
   for(int i=0; i<3; i++)
      for(int j=0; j<3; j++)
         a[i][j]=mul[i][j];
}

int power(int F[3][3],int n)
{
   int M[3][3]= {{1,1,1},{1,0,0},{0,1,0}};
   ///Multiply it with initial value i.e with
   ///F(0)=0,F(1)=1,F(2)=1
   if(n==1)
      return F[0][0]+F[0][1];
   power(F,n/2);
   multiply(F,F);
   if(n%2!=0)
      multiply(F,M);
   return F[0][0]+F[0][1];
}

int findNthTerm(int n)
{
```

```
   int F[3][3]= {{1,1,1},{1,0,0},{0,1,0}};
   return power(F,n-2);
}
```

Probability

Let, player one has n1 pennies and player two has n2 pennies

In the event of an unfair coin, where player one wins each toss with probability p, and player two wins with probability $q = 1 - p$, then the probability of each ending penniless is:

```
p1= (1 - (p/q) ^ n1) / (1 –(p/q)^(n1+n2) );
P2= (1 - (q/p) ^ n1) / (1 –(q/p)^(n1+n2) );
```

For fair coin

```
P1=n2/ (n1+n2);
P2=n1/(n1+n2);
```

Sum array sum

```
int ans = a[0], ans_l = 0, ans_r = 0;
int sum = 0, minus_pos = -1;

for (int r = 0; r < n; ++r) {
   sum += a[r];
   if (sum > ans) {
      ans = sum;
      ans_l = minus_pos + 1;
      ans_r = r;
   }
   if (sum < 0) {
      sum = 0;
      minus_pos = r;
   }
}
```

----------sagar----------

-*MO Algo****

```
struct data
{
   int x,y,pos;
}v[3000009];
int k,ara[300009];int sum=0,ans[200009];int cnt[1000009];
bool comp(data u,data u1)
{
   int a1=u.x/k,b1=u1.x/k; if(a1==b1)return u.y<u1.y;return a1<b1;
}
```

```
void add(int ind)
{
   int P=ara[ind]; cnt[P]++;if(cnt[P]==1)sum++;
}
void removes(int ind)
{
   int P=ara[ind];cnt[P]--;if(cnt[P]==0)sum--;
}
int main()
{
   int n;scanf("%d",&n);k=sqrt(n);for(int i=0;i<n;i++)scanf("%d",&ara[i]);
   int q;scanf("%d",&q); for(int i=0;i<q;i++){
      scanf("%d %d",&v[i].x,&v[i].y); v[i].x--; v[i].y--;v[i].pos=i;
   }
   sort(v,v+q,comp);
   int L=0,R=-1;
   for(int i=0;i<q;i++)
   {
      while(L<v[i].x)removes(L++);while(R<v[i].y)add(++R);while(L>v[i].x)add(--L);
      while(R>v[i].y)removes(R--);
      ans[v[i].pos]=sum;
   }
   for(int i=0;i<q;i++)printf("%d\n",ans[i]);return 0;
}
```

****Cycle Finding from undirected graph.***

if you don't want to find cycles,then just count the number of edges in each connected component, you can do it by summing up the degrees of all nodes in that component then divide it by 2. if the number of edges is equal to number of nodes minus 1. then it's tree and there's no cycle, otherwise there's a cycle.

Number of divisors*

```
     sum_of_divisors=1;
     num_of_divisors=1;
       for(i=2;i<=num;i++)
       {
         exponent=0;
          while(num%i==0)
           {
              exponent++;
              num=num/i;
           }
           num_of_divisors*=(exponent+1);
sum_of_divisors*=(pow(i,exponent+1)-1)/(i-1);
   }
```

----------sohel----------

Segment_Tree_Sum

```
const int N = (int)1e5 + 5;
int n;
ll seg[4 * N + 4];
ll lazy[4 * N + 4];

void push(int at, int l, int r) {
   seg[at] += lazy[at] * (r - l + 1);
   if(l < r) {
      lazy[2*at] += lazy[at];
      lazy[2*at+1] += lazy[at];
   }
   lazy[at] = 0;
}
void modify(int at, int l, int r, int x, int y, ll val) {
   if(l > y || r < x)
      return ;
   if(x <= l && r <= y)
      lazy[at] += val;
   else {
      seg[at] += val * (( min(r, y) - max(l, x) ) + 1 );
      int mid = (l + r) >> 1;
      modify(2 * at, l, mid, x, y, val);
      modify(at<<1|1, mid + 1, r, x, y, val);
   }
}
ll get(int at, int l, int r, int x, int y) {
   if(l > y || r < x) {
      return 0;
   }
   push(at, l, r);
   if(l >= x && r <= y) {
      return seg[at];
   }
   int mid = (l+r)>>1;
   return (get(at*2, l, mid, x, y) + get(at*2+1, mid+1, r, x, y));
}
```

Convex Hall

```
struct pt {
        int x, y;
};
int n, x, y;
vector < pt > a , conv;
int cross3(pt & a, pt & b , pt & c){
   return (b.x - a.x) * (c.y - b.y) - (c.x - b.x) * (b.y - a.y);
}
void hall () {
        int i, t, n = (int)a.size(), k = 0;
        conv.resize(2*n);
   sort (a.begin(), a.end(), [&] (const pt & p, const pt & q) {
      return make_pair (p.x, p.y) < make_pair (q.x, q.y);
   });
        for (i=0; i<n; i++) {
                while (k >= 2 && cross3(conv[k-2], conv[k-1], a[i]) <= 0) k--;
                conv[k++] = a[i];
        }
        for (i=n-1, t=k+1; i>=0; i--) {
                while (k >= t && cross3(conv[k-2], conv[k-1], a[i]) <= 0) k--;
                conv[k++] = a[i];
        }
   while(k > 1 && make_pair(conv[0].x , conv[0].y) == make_pair(conv[k-1].x, conv[k-1].y))
      k--;
        conv.resize(k);
}
```

Monotonic_RMQ

```
template < typename T , bool minimum_mode >
struct monotonic_rmq{
   deque<pair<T,int>>q;
   int cnt_added = 0;
   int cnt_removed = 0;
   bool good(const T & a, const T & b) const {
      return minimum_mode ? a < b : a > b;
   }
   T get(){
      assert(!q.empty());
```

```
        return q.front().first;
    }
    void push(T a){
        while(!q.empty() && !good(q.front().first , a)){
            q.pop_back();
        }
        q.push_back(make_pair(a, cnt_added));
        cnt_added++;
    }
    void pop(){
        if(!q.empty() && q.front().second == cnt_removed){
            q.pop_front();
        }
        cnt_removed++;
    }
};
```

Policy based DS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using o_set =
    tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
/// *s.find_by_order(k) -> returns the k'th smallest element (indexing starts from 0)
/// s.order_of_key(v)  -> returns how many elements are strictly smaller than v

o_set<int> s;
gp_hash_table<int , int>maps;
```

RANDOM

```
mt19937 rng((unsigned int)chrono::steady_clock::now().time_since_epoch().count());
```

Modulo Inverse

```
/// if m is prime then it  compute the modular inverse
///for every number in the range [1,m−1] in O(m)
/// it is generally used for small K such that we need up to K module inverse

inv[1] = 1;
for(int i = 2; i < md; ++i)
   inv[i] = (md - (md/i) * inv[md%i] % md) % md;
```

SIEVE_COMPRESS

```
#define size 100000100
long long mark[size / 64 + 100];
unsigned prime[5761558];
int sieve (int n) {
   int cnt = 0;
   for (long long i = 3; i * i <= n; i += 2)
      if (! (mark[i>>6]& (1LL<< (i&63))))
         for (long long j = i * i; j <= n; j += 2 * i)
            mark[j>>6] |= (1LL<< (j&63));
   prime[cnt++] = 2;
   for (long long i = 3; i <= n; i += 2)
      if (! (mark[i>>6] & (1LL << (i & 63))))
         prime[cnt++] = i;
   return cnt; }
```

<u>Chinese Remainder Theorem</u>

```
 ll egcd(ll a, ll b, ll & x, ll &y){
   if(a == 0){
      x = 0 ; y = 1 ; return b;
   }
   ll x1 , y1;
   ll g = egcd(b % a , a, x1, y1);
   x = y1 - (b / a) * x1;
   y = x1;
   return g;
}

ll invMod(ll n , ll mod){
   ll x, y , g;
   g = egcd(n , mod , x, y);
```

```
    assert(g == 1);
    return  (x + mod) % mod;
}
pair<ll , ll> CRC(vector<ll> a , vector<ll> m){
    assert(a.size() == m.size());
    int n = a.size();
    ll prod = 1LL;
    for(int i = 0; i < n; i++) prod *= m[i];
    ll res = 0LL;
    for(int i = 0; i < n; i++){
        ll he = prod / m[i];
        res += a[i] * he *  invMod(he , m[i]);
        res %= prod;
    }
    return make_pair((res + prod) % prod , prod);
}
```

Lucas Theorem

```
ll fact[maxn];

inline ll nCr (ll n, ll r, ll p)
{

    if (n < r)
        return 0;
    ll upor = fact[n] % p;
    ll nich = (fact[r] * fact[n-r]) % p;
    nich = invMod(nich , p) ;
    return (upor * nich) % p;
}

ll lucas (ll n, ll r, ll p)
{

    if (n == 0 and r == 0)
        return 1;
    ll ni = n % p;
    ll ri = r % p;
    return (nCr (ni, ri, p) * lucas (n/p, r/p, p)) % p;
}
```

```
inline ll get_nCr (ll n, ll r, ll p)
{
   fact[0] = 1;
   for (int i = 1; i < p; i++)
      fact[i] = (i * fact[i-1]) % p;
   return lucas (n, r, p);
}
```

Gaussian elimination

Ax + By + …….+ = c

```
using ld = long double;
vector<ld> solve(vector<vector<ld>> & a){
   int Row = (int)a.size();
   for(int k = 0; k < Row; k++){
      int pivot = k;
      ld mx = a[k][k];
      for(int i = k + 1; i < Row; i++){
         if(fabs(a[i][k]) > mx){
            mx = a[i][k];
            pivot = i;
         }
      }
      if(pivot != k){
         a[pivot].swap(a[k]);
      }

      for(int i = k + 1; i < Row; i++){
        ld alpha = a[i][k] / a[k][k];
         for(int j = k + 1; j <=Row; j++){
            a[i][j] -= a[k][j] * alpha;
         }
         a[i][k] = 0.0;
      }
   }
   vector<ld>answer(Row);
   for(int i = Row - 1; i >= 0; i--){
      answer[i] = a[i][Row];
      for(int j = i + 1; j < Row; j++){
         answer[i] -= a[i][j] * answer[j];
```

```
        }
        answer[i] /= a[i][i];
    }
    return answer;
}
```

LCA

```
int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
```

```
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
}
```

Heavy-light decomposition

The definition of **heavy edge** can be changed to **the edge leading to the child with largest subtree**, with ties broken arbitrarily. This may result is some light edges being converted to heavy, which means some heavy paths will combine to form a single path, but all heavy paths will remain disjoint. It is also still guaranteed that going down a light edge reduces subtree size to half or less.

Instead of a building segment tree over every heavy path, a single segment tree can be used with disjoint segments allocated to each heavy path.

It has been mentioned that answering queries requires calculation of the LCA. While LCA can be calculated separately, it is also possible to integrate LCA calculation in the process of answering queries.

To perform heavy-light decomposition:

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}
```

```
int decompose(int v, int h, vector<vector<int>> const& adj) {
   head[v] = h, pos[v] = cur_pos++;
   if (heavy[v] != -1)
      decompose(heavy[v], h, adj);
   for (int c : adj[v]) {
      if (c != parent[v] && c != heavy[v])
         decompose(c, c, adj);
   }
}

void init(vector<vector<int>> const& adj) {
   int n = adj.size();
   parent = vector<int>(n);
   depth = vector<int>(n);
   heavy = vector<int>(n, -1);
   head = vector<int>(n);
   pos = vector<int>(n);
   cur_pos = 0;

   dfs(0, adj);
   decompose(0, 0, adj);
}
```

The adjacency list of the tree must be passed to the **init** function, and decomposition is performed assuming vertex **0** as root.
The **dfs** function is used to calculate **heavy[v]**, the child at the other end of the heavy edge from **v**, for every vertex **v**. Additionally **dfs** also stores the parent and depth of each vertex, which will be useful later during queries.
The **decompose** function assigns for each vertex **v** the values **head[v]** and **pos[v]**, which are respectively the head of the heavy path **v** belongs to and the position of **v** on the single segment tree that covers all vertices.
To answer queries on paths, for example the maximum query discussed, we can do something like this:

```
int query(int a, int b) {
   int res = 0;
   for (; head[a] != head[b]; b = parent[head[b]]) {
      if (depth[head[a]] > depth[head[b]])
         swap(a, b);
      int cur_heavy_path_max = segment_tree_query(pos[head[b]], pos[b]);
      res = max(res, cur_heavy_path_max);
   }
   if (depth[a] > depth[b])
      swap(a, b);
```

```
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

Intersection Point of Lines

```
struct pt {
    double x, y;
};

struct line {
    double a, b, c;
};

const double EPS = 1e-9;

double det(double a, double b, double c, double d) {
    return a*d - b*c;
}

bool intersect(line m, line n, pt & res) {
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS)
        return false;
    res.x = -det(m.c, m.b, n.c, n.b) / zn;
    res.y = -det(m.a, m.c, n.a, n.c) / zn;
    return true;
}

bool parallel(line m, line n) {
    return abs(det(m.a, m.b, n.a, n.b)) < EPS;
}

bool equivalent(line m, line n) {
    return abs(det(m.a, m.b, n.a, n.b)) < EPS
        && abs(det(m.a, m.c, n.a, n.c)) < EPS
        && abs(det(m.b, m.c, n.b, n.c)) < EPS;
}
```

Closest pair points

$O(nlog(n))$. Code is below:

```
long long ClosestPair(vector<pair<int, int>> pts) {
```

```
    int n = pts.size();
    sort(pts.begin(), pts.end());
    set<pair<int, int>> s;

    long long best_dist = 1e18;
    int j = 0;
    for (int i = 0; i < n; ++i) {
        int d = ceil(sqrt(best_dist));
        while (pts[i].first - pts[j].first >= best_dist) {
            s.erase({pts[j].second, pts[j].first});
            j += 1;
        }

        auto it1 = s.lower_bound({pts[i].second - d, pts[i].first});
        auto it2 = s.upper_bound({pts[i].second + d, pts[i].first});

        for (auto it = it1; it != it2; ++it) {
            int dx = pts[i].first - it->second;
            int dy = pts[i].second - it->first;
            best_dist = min(best_dist, 1LL * dx * dx + 1LL * dy * dy);
        }
        s.insert({pts[i].second, pts[i].first});
    }
    return best_dist;
}
```

Check if point belongs to the convex polygon in O(logN)

```
struct pt{
    long long x, y;
    pt(){}
    pt(long long _x, long long _y):x(_x), y(_y){}
    pt operator+(const pt & p) const { return pt(x + p.x, y + p.y); }
    pt operator-(const pt & p) const { return pt(x - p.x, y - p.y); }
    long long cross(const pt & p) const { return x * p.y - y * p.x; }
    long long dot(const pt & p) const { return x * p.x + y * p.y; }
    long long cross(const pt & a, const pt & b) const { return (a - *this).cross(b - *this); }
    long long dot(const pt & a, const pt & b) const { return (a - *this).dot(b - *this); }
    long long sqrLen() const { return this->dot(*this); }
};

bool lexComp(const pt & l, const pt & r){
    return l.x < r.x || (l.x == r.x && l.y < r.y);
```

```
}

int sgn(long long val){
   return val > 0 ? 1 : (val == 0 ? 0 : -1);
}

vector<pt> seq;
int n;

bool pointInTriangle(pt a, pt b, pt c, pt point){
   long long s1 = abs(a.cross(b, c));
   long long s2 = abs(point.cross(a, b)) + abs(point.cross(b, c)) + abs(point.cross(c, a));
   return s1 == s2;
}

void prepare(vector<pt> & points){
   n = points.size();
   int pos = 0;
   for(int i = 1; i < n; i++){
      if(lexComp(points[i], points[pos]))
         pos = i;
   }
   rotate(points.begin(), points.begin() + pos, points.end());

   n--;
   seq.resize(n);
   for(int i = 0; i < n; i++)
      seq[i] = points[i + 1] - points[0];
}

bool pointInConvexPolygon(pt point){
   if(seq[0].cross(point) != 0 && sgn(seq[0].cross(point)) != sgn(seq[0].cross(seq[n - 1])))
      return false;
   if(seq[n - 1].cross(point) != 0 && sgn(seq[n - 1].cross(point)) != sgn(seq[n - 1].cross(seq[0])))
      return false;

   if(seq[0].cross(point) == 0)
      return seq[0].sqrLen() >= point.sqrLen();

   int l = 0, r = n - 1;
   while(r - l > 1){
      int mid = (l + r)/2;
      int pos = mid;
      if(seq[pos].cross(point) >= 0)l = mid;
```

```
    else r = mid;
 }
 int pos = l;
 return pointInTriangle(seq[pos], seq[pos + 1], pt(0, 0), point); }
typedef
unsigned
long
long ull;
                const int MN  = 2;
                struct Matrix {
                   int r, c;
                   ull mat[MN][MN];
                   Matrix (int _r, int _c) : r (_r), c (_c) {
                      memset (mat, 0, sizeof mat);
                   }
                   ull x[MN][MN];
                   Matrix & operator *= (const Matrix &o) {
                      memset (x, 0, sizeof x);
                      for (int i = 0; i < r; ++i)
                         for (int k = 0; k < c; ++k)
                            if (mat[i][k] != 0)
                               for (int j = 0; j < c; ++j)
                                  x[i][j] += mat[i][k] * o.mat[k][j];
                      memcpy (mat, x, sizeof (mat));
                      return *this;
                   }
                };
                void power (Matrix b, long long e, Matrix &res) {
                   memset (res.mat, 0, sizeof res.mat);
                   for (int i = 0; i < b.r; ++i)
                      res.mat[i][i] = 1;
                   while (e > 0) {
                      if (e & 1)
                         res *= b;
                      b *= b;
                      e >>= 1;
                   }
                }
                void solve() {
                   ull p, q, n;
                   scanf ("%llu%llu%llu", &p, &q, &n);
                   if (n == 0) {
                      printf ("2\n");
                      return;
```

```
        }
        Matrix mat (2, 2);
        mat.mat[0][0] = p;
        mat.mat[0][1] = -q;
        mat.mat[1][0] = 1;
        mat.mat[1][1] = 0;
        Matrix ans (2, 2);
        power (mat, n - 1, ans);
        printf ("%llu\n", ans.mat[0][0] * p + ans.mat[0][1] * 2);
    }
```

```
struct
Edge
{
        int to, next;
        Edge (int _to = 0, int _next = -1) :to (_to), next (_next) {}
    };
    const int N = 1e5;
    Edge e[N * 2];
    int head[N], tin[N], low[N], timer, tot ;
    int n, m, ans;
    bool vis[N] ;
    void add_edge (int from, int to) {
        e[tot] = Edge (to, head[from]);
        head[from] = tot++;
        e[tot] = Edge (from, head[to]);
        head[to] = tot++;
    }
    void all_clear() {
        timer = tot = ans = 0;
        for (int i = 0; i< N; ++i) {
            tin[i] = low[i] = vis[i] = 0;
            head[i] = -1;
        }
    }
    void Dfs (int u, int par) {
        tin[u] = low[u] = ++timer;
        int bacca = 0;
        for (int i = head[u]; ~i; i = e[i].next) {
            int to = e[i].to;
            if (tin[to] == 0) {
                Dfs (to, u);
```

```
            low[u] = min (low[u], low[to]);
            bacca++;
            if (!vis[u] && ( (par==-1 && bacca > 1) || (par != -1 && tin[u] <= low[to]))) {
              vis[u] = 1;
              ans++;
            }
          } else
            low[u] = min (low[u], tin[to]);
        }
      }
      void test_case (int kase) {
        cout << "Case " << kase << ": ";
        cin >> n >> m;
        all_clear();
        for (int i = 0; i < m; ++i) {
          int from, to ;
          cin >> from >> to;
          add_edge (from, to);
        }
        Dfs (1, -1);
        cout<< ans << '\n';
      }
      int main() {
      #ifdef LOCAL
        freopen ("input.txt", "r",  stdin);
      //   freopen("output.txt", "w", stdout);
      #endif
        ios::sync_with_stdio (false);
        cin.tie (0);
        int tt;
        cin >> tt;
        for (int t = 1; t<=tt; t++)
          test_case (t);
        return 0;
      }
```